GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Module M8: LED Animation and Physical Board

Instructor: Erkan Zergeroğlu **Company:** Board&Byte **Group:** Group-8 **Date:** 6 November 2026

Name	Student No.	Name	Student No.
Ayşe Poyraz	230104004085	Hasan Hüseyin Özdemir	230104004048
Muhib Taha Boy	230104004165	Umutcan Ocak	220104004903

1. Purpose

M8 builds the physical city board and writes the animation engine that turns the game state into light. The board is 60 × 60 cm. It has 344 addressable LEDs on the roads and in the stores, 3D-printed buildings, and places for the motion sensor and the vibration motors. In the game the board is the only map, so it must be clear to read from every side of the table.

Runs on	Core 1 of the ESP32 for the software; the board itself for the hardware
Technology	Portable C++ animation engine that runs on the ESP32 and on a PC for tests; 344 addressable RGB LEDs; level shifter; 3D printing; plywood base
Requirements	R2 R5 R6 R7 R8 R11

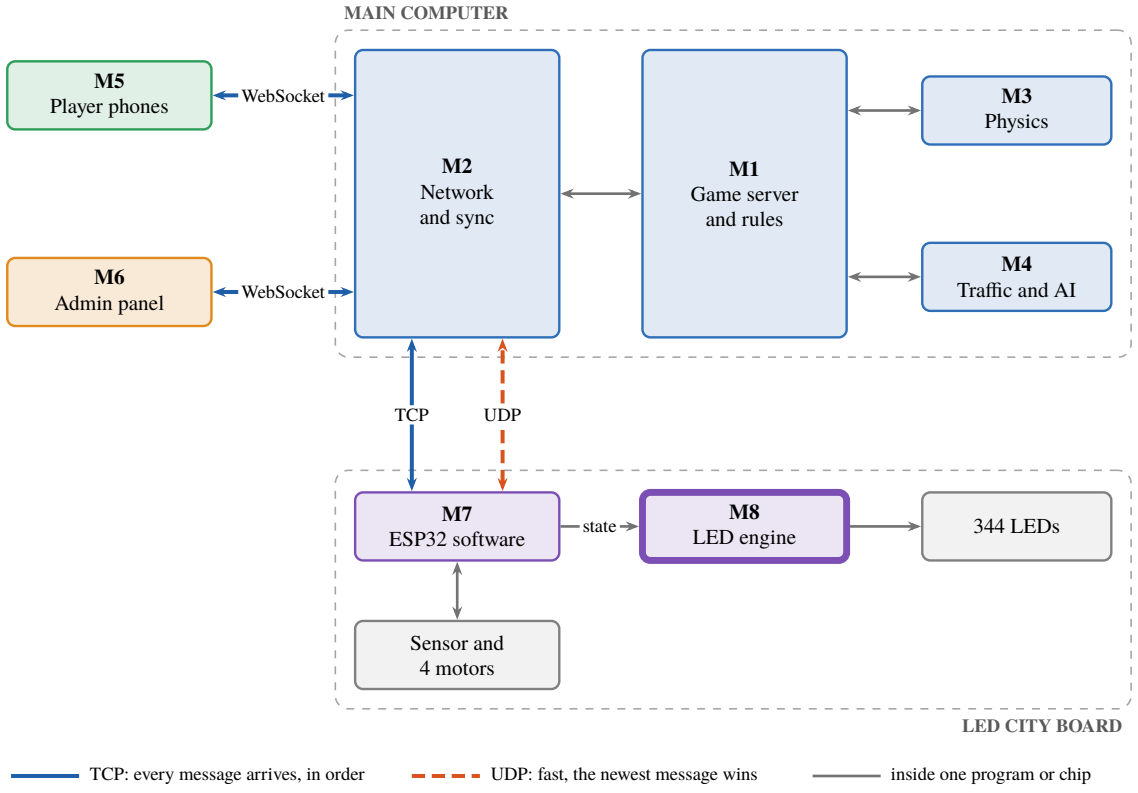


Figure 1: Where M8 sits in the system. M8 gets the state from M7 on the same chip.

2. Requirements covered

ID	Requirement	What M8 does for it
R2	Delivery cycle	Delivery points blink in the player's tone; a flash when the package is delivered
R5	No map on the phone	The board is the map: all motorcycles; "Find me" white flash for 1 s
R6	Obstacles	Red patterns by type on the whole road segment
R7	Earthquake	A shock wave from the earthquake centre

ID	Requirement	What M8 does for it
R8	Physical LED board	60 × 60 cm, 6 roads in each direction, LEDs on all roads, printed buildings, four stores
R11	Independent development	Demo mode without a server; animations checked as images on a PC

3. Inputs and outputs

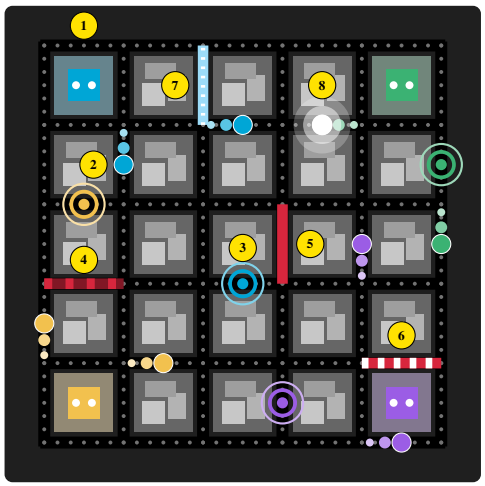
Dir.	Data	From / to	Content
In	Board state	M7	At most 30 per second: phase, players, delivery points, obstacles, earthquake
In	Config	M7	Player colours, brightness 0.1–1, current limit
In	Link state	M7	Connected, searching or lost
In	City plan	Shared file	Road grid and store places, turned into the LED map when the code is built
Out	Light	Board	60 frames per second
Out	Frame report	M7	Frame time and current estimate, shown to the admin

4. Main components

Component	What it does	Related modules
Motion smoother	Moves the motorcycle dots smoothly between two states	M7
LED map	The place of every LED; made from the city plan by a script	M1, city plan
Layers and effects	Five layers and the patterns for obstacles, earthquakes and game phases	—
Post-processing and output	Colour correction, brightness, current limit; drives 5 LED lines at the same time	M7
Demo mode	A built-in scenario that runs without a server	—

5. How it works

The board. The base is plywood. On it there are 6 roads in each direction, so the city has 5 × 5 blocks. LED strips with 60 LEDs per metre lie in the roads. A road segment between two crossings is 10 cm long, so it has exactly 6 LED steps. With 336 LEDs on the roads and 2 LEDs in each of the 4 stores, the board has 344 LEDs. The buildings are 3D-printed from the same models as the 3D city on the phone, at a scale of 1 m in the game to 1 mm on the board. Thin printed covers over the roads soften the light. The LEDs need 5 V. The software limits the current to 4 A, so the power supply never gets too hot. A level shifter turns the 3.3 V signal of the ESP32 into a clean 5 V signal.



1 Team store: two LEDs in team colour
2 Motorcycle: a bright dot with a short tail
3 Delivery point: blinks in the player's tone
4 Heavy traffic: a slow dim red wave
5 Road work: bright steady red
6 Accident: red and white blinking
7 Water puddle: a cool white shimmer
8 "Find me": white flash for 1 second

Figure 2: The planned LED board from above, during a game with 8 players.

Colours. There are four teams and each team has one colour: blue, green, yellow and purple. The two players of a team have two tones of the team colour, a light one and a dark one. Red is used only for obstacles, so red always means "this road slows you down". The team colours are far from red, so nobody mixes them up. The pattern of the red tells the type of the obstacle, as the figure below shows.

Layers. The engine draws 60 frames per second on core 1. Each frame is made from five layers, from the bottom to the top: the roads, the obstacles, the delivery points, the motorcycles and a top layer for short effects. A higher layer covers a lower one. The board state comes only 10 to 30 times per second, so the motion smoother moves each motorcycle dot between the last two states. The dot has a short tail, so the direction of motion is easy to see. The top layer draws the "Find me" flash and the earthquake shock wave. After the layers, the engine corrects the colours for the eye, sets the brightness from the config,

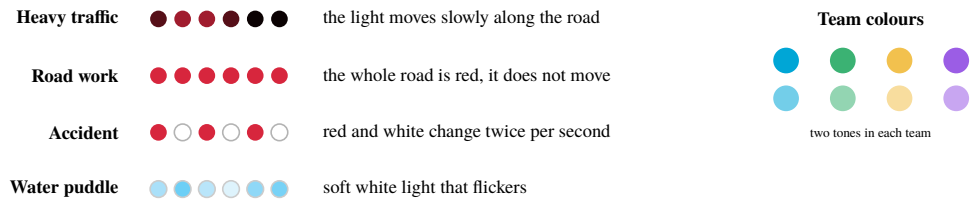


Figure 3: Left: the six LEDs of one road segment for each obstacle type. Right: the team colours.

checks the current and lowers all LEDs a little if the current would be over the limit. Then it sends the frame to the LEDs on 5 lines at the same time, so one frame takes only about 2 ms to send.

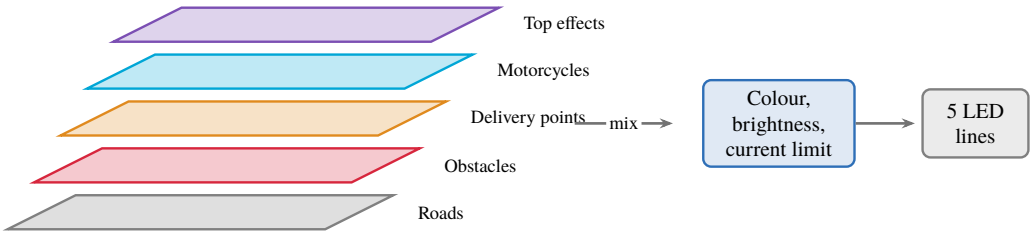


Figure 4: The five layers of one frame and the steps before the LEDs.

Earthquake and link loss. When an earthquake starts, a ring of light grows from the centre of the earthquake over the whole board, like a wave on water, and slowly fades. The roads inside the earthquake radius then turn red for road work. If M7 says that the link is lost, the roads go dark and the store LEDs breathe slowly in amber, so everybody can see that the board is waiting.

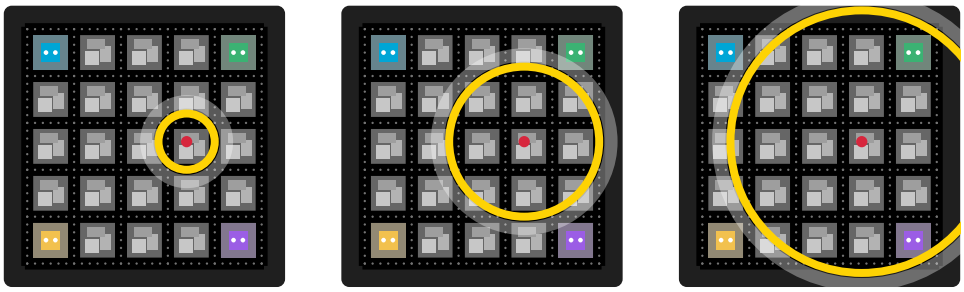


Figure 5: The earthquake shock wave grows from the centre of the hit.

6. Testing and demonstration

The engine is plain C++, so it also runs on a PC. There, every animation is drawn as an image and compared with a saved reference image. On the real board, built-in test patterns check the LED order and place, the 8 player colours (three people must name each colour correctly), the current limit, the voltage at the farthest LED, and 30 minutes of heat. In the demonstration the board runs its demo scenario without a server, with all obstacle types and an earthquake.